



Operating Systems Lab

Course Code: CSE-412

Experiment Title: Deadlock Avoidance using Banker's Algorithm

Student Name: Md. Shoyaif Rahman

ID: 0432310005101050

Instructor Name: Afroja Ahmed Smrity

University Name: University of Information Technology and Sciences
(UITS)

Experiment No: 05**Experiment Title:** Implementation of Banker's Algorithm for Deadlock Avoidance**Date of Performance:** 05/05/26**Date of Submission:** 05/05/26**Objective of the Experiment**

- To understand the concept of safe and unsafe system states.
- To implement the Banker's Algorithm to avoid deadlocks.
- To compute the Need matrix and determine a safe execution sequence.
- To analyze the system's safety given available resources.

Theory / Background

The Banker's Algorithm is a deadlock avoidance algorithm. It tests whether granting a resource request will lead to a safe state. A safe state is one where there exists at least one sequence in which all processes can complete without deadlocking. The algorithm uses the following data structures:

- Available: Number of available instances of each resource.
- Max: Maximum demand of each process.
- Allocation: Resources currently allocated to each process.
- Need: Remaining resources each process may need (Need = Max - Allocation).

Procedure

1. Input the number of processes (n) and resource types (m).
2. Input the Allocation matrix, Max matrix, and Available vector.
3. Calculate the Need matrix ($\text{Need}[i][j] = \text{Max}[i][j] - \text{Allocation}[i][j]$).
4. Initialize a finish[] array to false and an empty safeSeq[].
5. Find a process that is not finished and whose Need is less than or equal to Available.
6. If found, add its resources to Available, mark it as finished, and add it to safeSeq.
7. Repeat step 5-6 until all processes are finished (safe state) or no such process is found (unsafe state).
8. Print the safe sequence or an unsafe message.

Program Code:

```
#include <stdio.h>
#define MAX 10
int main() {
    int n, m;
    int alloc[MAX][MAX], max[MAX][MAX], need[MAX][MAX];
    int available[MAX], finish[MAX] = {0}, safeSeq[MAX];
    printf("Enter number of processes: "); scanf("%d", &n);
    printf("Enter number of resource types: "); scanf("%d", &m);
    printf("\nEnter Allocation Matrix:\n");
    for(int i = 0; i < n; i++) for(int j = 0; j < m; j++) scanf("%d", &alloc[i][j]);
    printf("\nEnter Max Matrix:\n");
    for(int i = 0; i < n; i++) for(int j = 0; j < m; j++) scanf("%d", &max[i][j]);
```

```

printf("Enter Available Resources: ");
for(int i = 0; i < m; i++) scanf("%d", &available[i]);

for(int i = 0; i < n; i++) for(int j = 0; j < m; j++) need[i][j] = max[i][j] - alloc[i][j];

int count = 0;
while(count < n) {
    int found = 0;
    for(int p = 0; p < n; p++) {
        if(!finish[p]) {
            int canAllocate = 1;
            for(int j = 0; j < m; j++) if(need[p][j] > available[j]) { canAllocate = 0; break; }
            if(canAllocate) {
                for(int j = 0; j < m; j++) available[j] += alloc[p][j];
                safeSeq[count++] = p;
                finish[p] = 1;
                found = 1;
            }
        }
    }
    if(!found) { printf("\nSystem is NOT in safe state.\n"); return 0; }
}
printf("\nSystem is in SAFE state.\nSafe sequence: ");
for(int i = 0; i < n; i++) printf("P%d ", safeSeq[i]);
printf("\n");
return 0;
}

```

Input and Output:

```

Enter number of processes: 5
Enter number of resource types: 3
Allocation:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Max:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Available: 3 3 2

```

Output:

```

System is in SAFE state.
Safe sequence: P1 P3 P4 P0 P2

```

Analysis / Discussion

The Banker's Algorithm correctly identified a safe state for the initial input. The Need matrix was calculated, and a valid sequence (P1 -> P3 -> P4 -> P0 -> P2) was found. When the Available vector was set to [0,0,0], the algorithm correctly identified an unsafe state, as no process's Need could be satisfied. A key insight is that an unsafe state is not necessarily a deadlock, but it could lead to one. The Banker's Algorithm prevents the system from entering such a risky state by denying requests that would lead to unsafety. This is a proactive, avoidance strategy.

9. Conclusion

The Banker's Algorithm was successfully implemented as a deadlock avoidance mechanism. It demonstrated how to determine safe and unsafe states by simulating resource allocation. The algorithm prevents deadlocks by ensuring the system always remains in a safe state, making it a powerful tool for resource management in operating systems.